FDE ACADEMY · LESSON 1 OF 9

Business Problem → Enterprise AI Architecture

Before writing a prompt, decide where AI belongs, and where it must not.

Raunak Sharan
Enterprise Agentic AI Engineering

Capstone: Agentic Credit Underwriting Platform

OBJECTIVES

By the end of today you can

01

Turn an observed workflow into requirements

02

Classify each step as AI, deterministic or human, and justify it

03

Draw service boundaries as typed contracts

04

Design for failure: what happens when a dependency returns nothing

05

Put a human in the loop deliberately, with machine-readable reasons

06

Run and extend a deterministic baseline that AI must beat

THE FDE’S FIRST WEEK

Meridian Digital Lending (fictional) says:

“We want AI in underwriting.”

That is not a requirement. Your job is to find the real one by watching the people who do the work today.

~45 min
per application, mostly gathering and re-keying

40 pages
of lending policy, searched by hand

Volume ↑
faster than the team can grow

MAP THE WORKFLOW

Today’s process, step by step, as observed

1. Application and documents arrive
2. Read payslips, re-key income
3. Pull bureau report
4. Calculate DTI by hand
5. Search the policy PDF
6. Draft decision memo; senior signs off
7. Paste into loan system

Where time goes: steps 2, 4 and 5 (highlighted)

Where errors come from: re-keying (2), stale policy knowledge (5), inconsistent notes (6–7)

WORKFLOW → REQUIREMENTS

The non-functional requirements decide the architecture

Requirement	Target	Built in
Never invent data (scores, income)	0 occurrences	Lessons 1, 6
Every decision cites evidence	100%	Lessons 3, 6
Survives a bureau outage	Escalates, never guesses	Lessons 6, 8
Decision replayable from audit trail	Every case	Lessons 7, 9
End-to-end latency	p95 under 15 s	Lessons 7, 8
Cost per application	Tracked per decision	Lesson 7
Role-based access	Underwriter, reviewer, admin	Lesson 9

THE ONE-QUESTION TEST · ADR-0001

Does this step need judgement, or does it need to be exactly right every time?

Judgement → AI

Reading messy documents

Finding relevant policy prose

Weighing evidence, writing a rationale

Choosing which tool to call next

Exactness → code

DTI, payments, ratios

Applying numeric thresholds

Validation and identity matching

Recording the audit trail

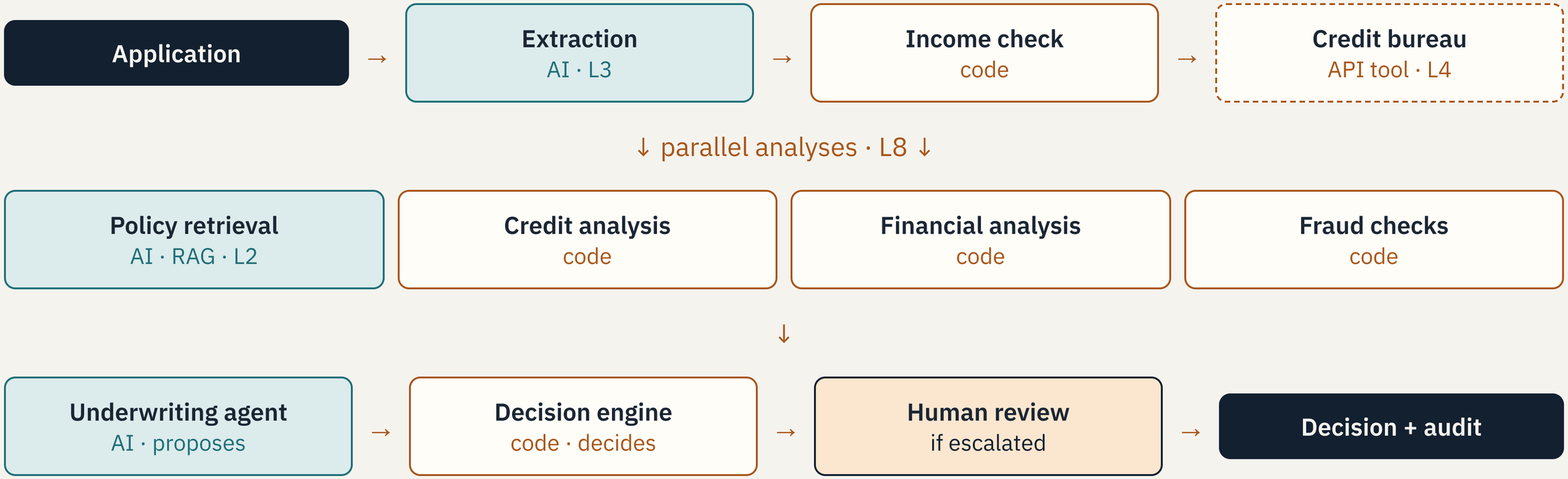
APPLY THE TEST

Most of the system is ordinary code

Component	Built as	Because
Document extraction	AI · lesson 3	Real documents are messy and varied
Income verification	Code	A rule with exact thresholds
Credit bureau	API tool	System of record; never estimated
Policy retrieval	AI · lesson 2	Long prose; reviewers need citations
Credit, financial, fraud checks	Code	Arithmetic and comparisons
Underwriting agent	AI · lessons 3–5	Judgement and explanation
Decision engine	Code	Policy is a commitment
Exceptions	Human	Required by policy

THE ARCHITECTURE

AI where it earns its place, code everywhere else



BOUNDARIES ARE TYPED CONTRACTS

Proposal and decision are different types

```
class AgentRecommendation(BaseModel):
recommendation: Recommendation
rationale: str
risk_factors: list[str]
missing_information: list[str]
evidence: list[Evidence]

class UnderwritingDecision(BaseModel):
recommendation: Recommendation
reason_codes: list[ReasonCode]
agent_recommendation: AgentRecommendation
overridden_by_policy: bool
requires_human_review: bool
audit_trail: list[AuditEvent]
```

Swap, don’t rewrite

Today a rules stub fills **AgentRecommendation**. In lesson 3 an LLM does. Nothing else changes.

Every claim has a source

Evidence points to a document, bureau record or policy section.

ADR-0002

The agent may escalate. It may never relax policy.

Policy says	Agent says	Final	Overridden?
Approve	Approve	Approve	No
Approve	Manual review	Manual review	No · escalation
Approve	Decline	Manual review	Yes
Manual review	Approve	Manual review	Yes
Decline	Approve or review	Decline	Yes

Every override is logged. A rising override rate means the agent and the policy have drifted apart: that becomes an evaluation signal in lesson 6.

DESIGN FOR FAILURE

For every component: nothing, garbage, or never returns?

Failure	If unhandled	Designed response	Case
Bureau has no record	Agent assumes a score	Typed error → review	APP-003
Required document missing	DTI on stated income	DTI not computed → review	APP-007
Stated vs verified income	Over-lending	Gap above 15% → review	APP-004
Name mismatch on payslip	Identity fraud approved	Review, never auto-decline	APP-006
Agent relaxes policy	Policy breach	Engine overrides, logged	APP-008
Malformed LLM output	Crash or silent garbage	Validate, retry, escalate	Lesson 6

ADR-0003 · ESCALATE, NEVER GUESS

APP-003: the bureau has no record

```
$ underwriter run APP-003.json

═══ APP-003: MANUAL_REVIEW ═══
Reasons: CREDIT_DATA_UNAVAILABLE

[intake] Received APP-003
[income] Verified 6500.0
[credit_bureau] Unavailable
[analysis] credit: 1 signal
[agent] Proposed MANUAL_REVIEW
[decision_engine] MANUAL_REVIEW
[human_review] REV-APP-003
```

The tool raises **CreditDataUnavailable**; it never returns a default.

credit_score is absent from the metrics, and a test asserts that absence.

The reviewer sees exactly what is missing.

THE DETERMINISTIC BASELINE

It runs end to end today, with no LLM

Case	Scenario	Outcome	Reason
APP-001	Clean, well-qualified	Approve	Meets criteria
APP-002	Post-loan DTI 45.6%	Manual review	DTI band
APP-003	No bureau record	Manual review	Credit unavailable
APP-004	Stated income 43% above payslip	Manual review	Income mismatch
APP-005	Score 561, four delinquencies	Decline	Score, conduct
APP-006	Payslip name ≠ applicant	Manual review	Identity
APP-007	Self-employed, no tax return	Manual review	Missing document
APP-008	Agent approves at 43.9% DTI	Manual review	Engine override

LIVE WALKTHROUGH

APP-008: the engine overrides the agent

```
$ uv run underwriter run data/applications/APP-008.json

═══ APP-008: MANUAL_REVIEW ═══
Reasons: DTI_ABOVE_REVIEW_THRESHOLD
Agent proposed APPROVE; overridden by policy
post_loan_dti=0.4391 credit_score=705

[agent] Proposed APPROVE: affordability looks sound
[decision_engine] Decided MANUAL_REVIEW (overrode agent)
[human_review] Escalated as REV-APP-008
```

The stub agent uses a 45% DTI cut-off; policy §4.2 says 43%. An LLM will also be approximately right. The architecture must not depend on it being exactly right.

LAB · 90 MINUTES

Three exercises

20 MIN · WORKSHEET

Classify the workflow

For each of the 7 steps: AI, code or human; why; and what happens if it fails. Find one step where reasonable people disagree and argue both sides.

40 MIN · CODE

Add a policy rule end to end

Policy §2.1: applicants must be 21 or older. Reason code, threshold, analyzer metric, engine rule, boundary test, sample case APP-009.

Done when tests pass and run-all shows 9/9

30 MIN · WRITING

Write ADR-0004

Pick one: should an LLM read thresholds from retrieved policy text? Should a fraud signal ever auto-decline? Context, decision, consequences.

HOMEWORK · SUBMIT AS A PULL REQUEST

What strong work looks like

Criterion	Strong	Weak
AI vs code reasoning	Justified by exactness, audit, failure impact	“LLMs are good at this”
Boundaries	New logic behind an existing contract	Logic leaks into orchestrator
Failure handling	Explicit reason codes; nothing guessed	Defaults, silent fallbacks
Tests	Both sides of every threshold	Happy path only
Writing	An ADR a new engineer can act on	Restates the code

The PR contains: exercise 2 with tests · your ADR-0004 · one new row in failure-modes.md

TAKEAWAY

The scaffold is what makes the AI safe to ship.

Next · Lesson 2

Enterprise RAG: the Credit Policy Intelligence Service

We replace the static policy map with hybrid retrieval, and measure whether it cites the right sections.